

Lecture 25: Machine Learning Continued

Heather Mattie, PhD

December 2, 2025

Recipe of the Day!

Korean Fried Cauliflower



Chef Emre

Announcements

- Lab this week on machine learning
- Homework #4 due Friday, December 5th
- Homework #5 due Friday, December 12th

Machine Learning

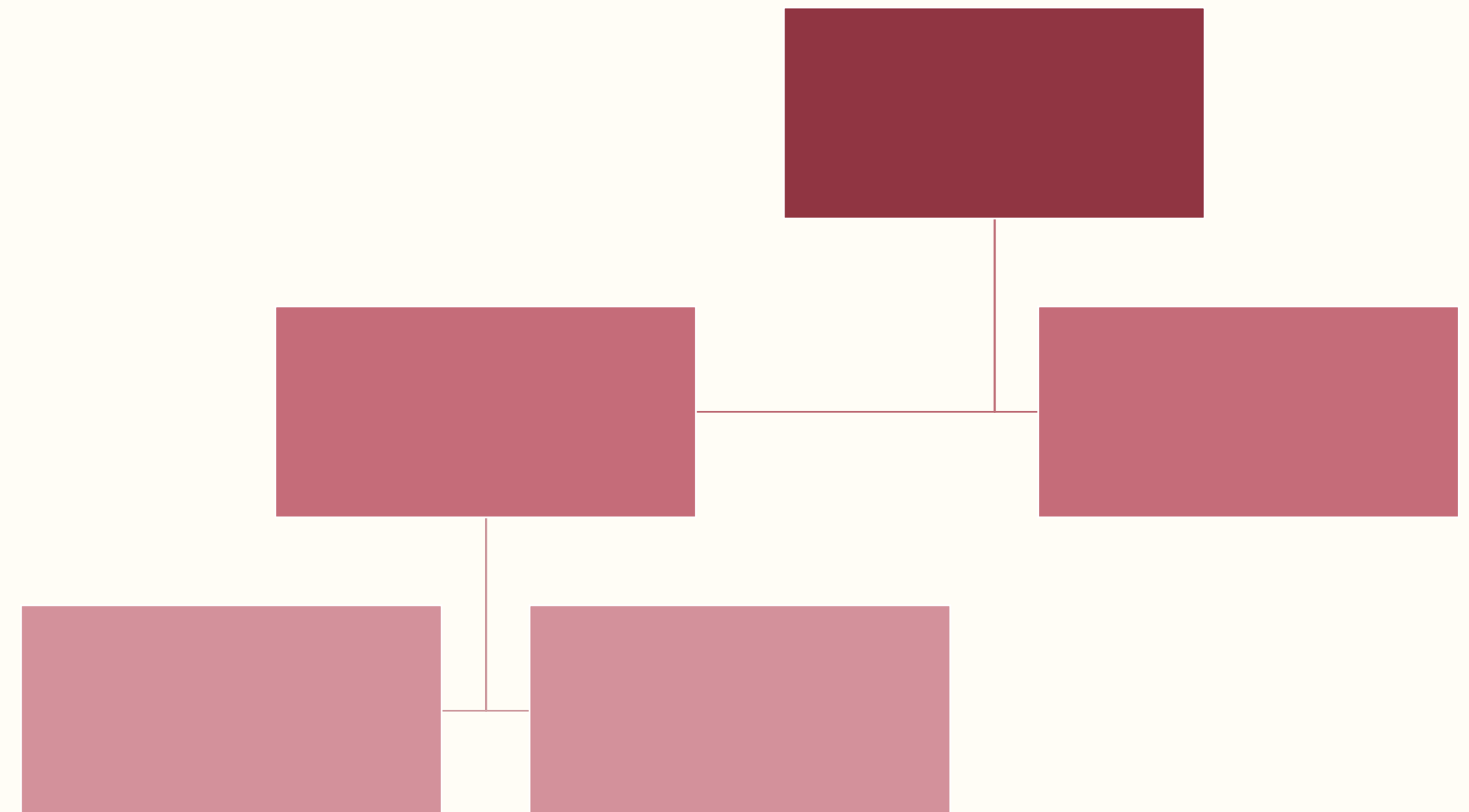
Decision trees

Heather Mattie, PhD

Decision trees

A **decision tree** is a simple model that makes predictions by asking a series of yes/no questions, similar to a flowchart. The model recursively splits data into smaller, more homogeneous groups.

- Interpretable
- Mimics “if-else” logic
- Common in healthcare



Training

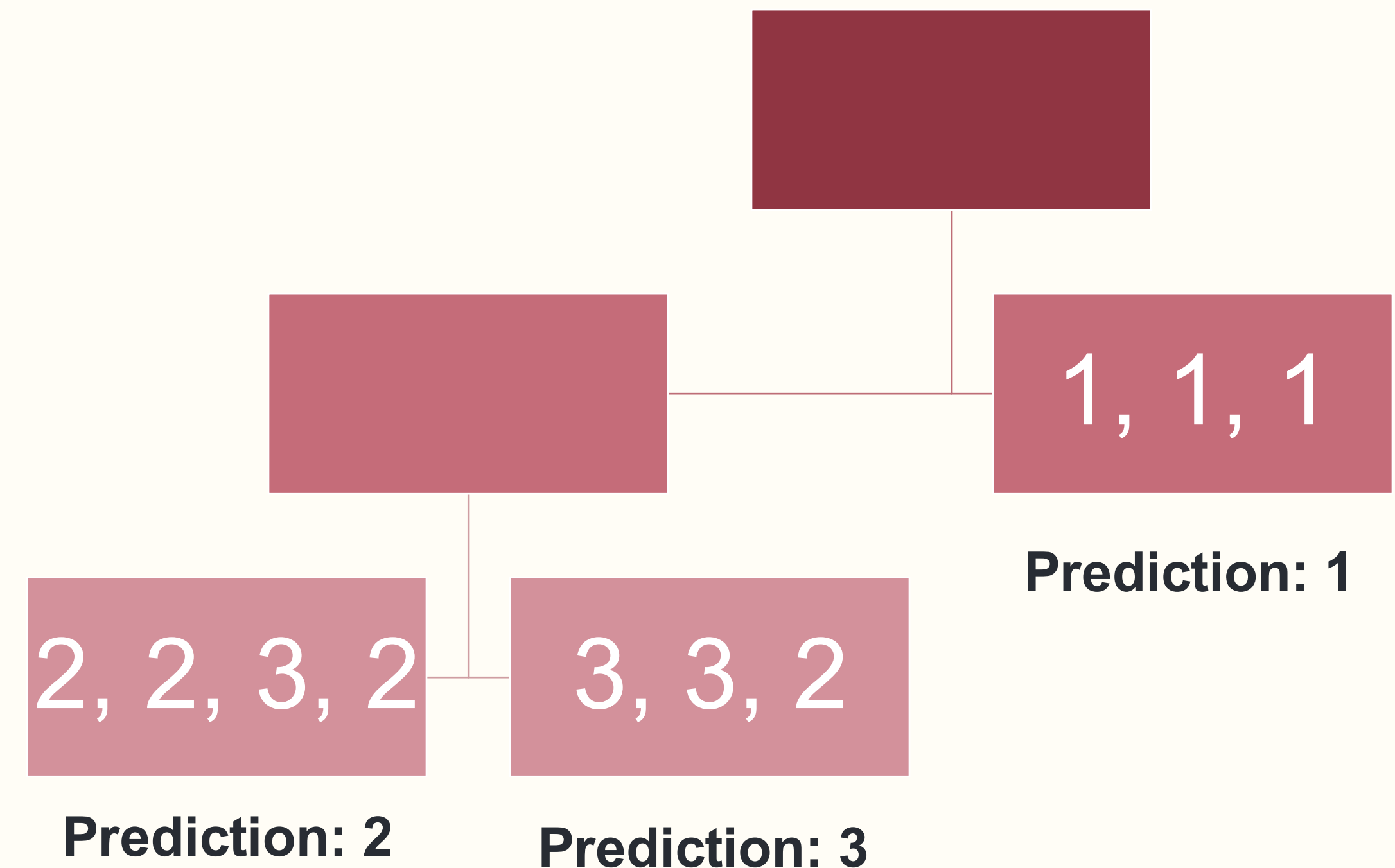
Steps:

1. Start with the entire dataset
2. Ask a yes/no question based on a predictor (e.g., “Is age < 50?”)
3. Split the data according to the answer
4. Continue splitting until the final groups (leaves/buckets) are consistent (homogeneous) and small

Making predictions

The way decision trees make predictions depends on the type of task.

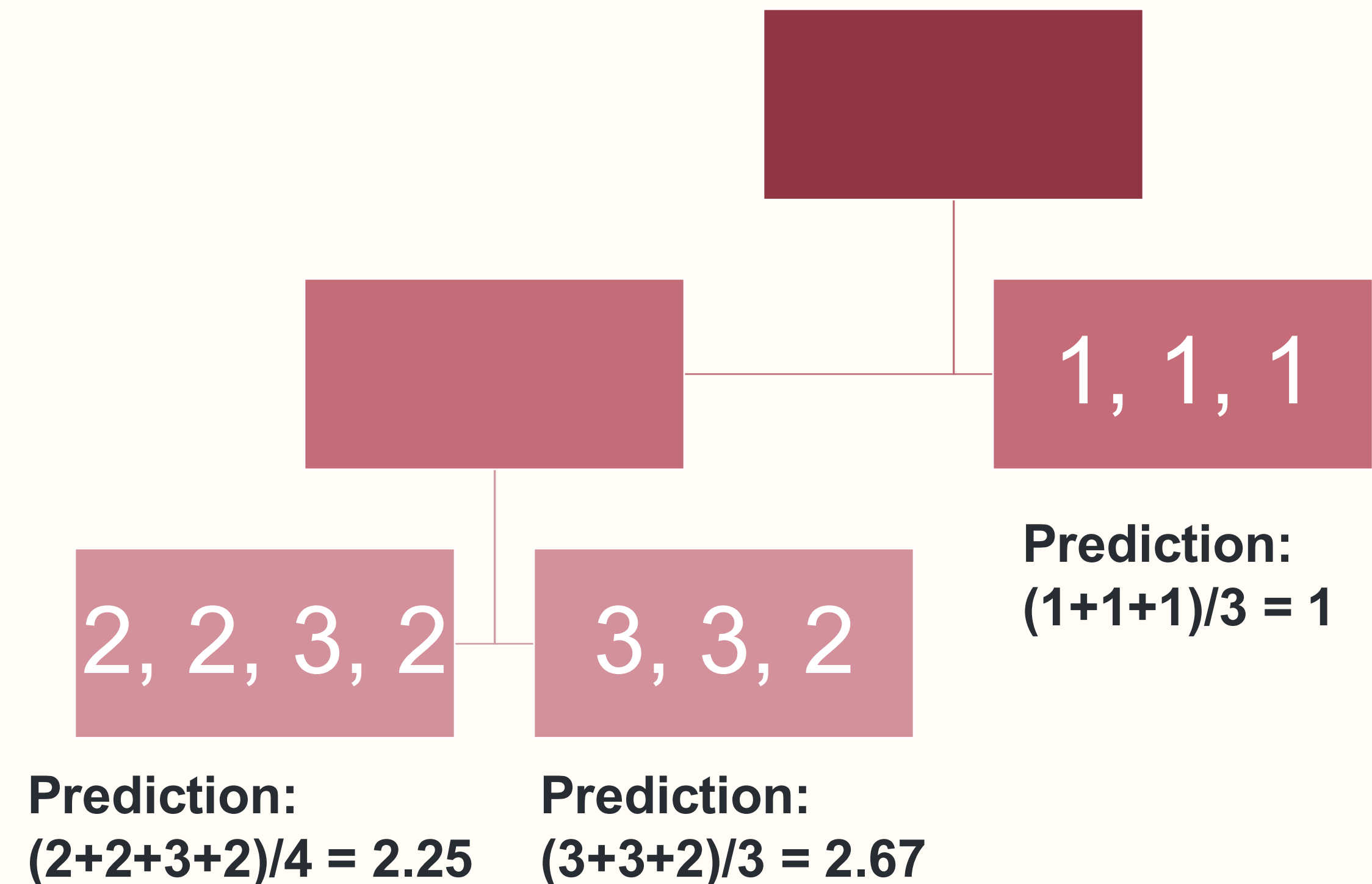
- Classification: predicts the **most common category** in the final group/leaf
- Regression: predicts the average value in the final group/leaf



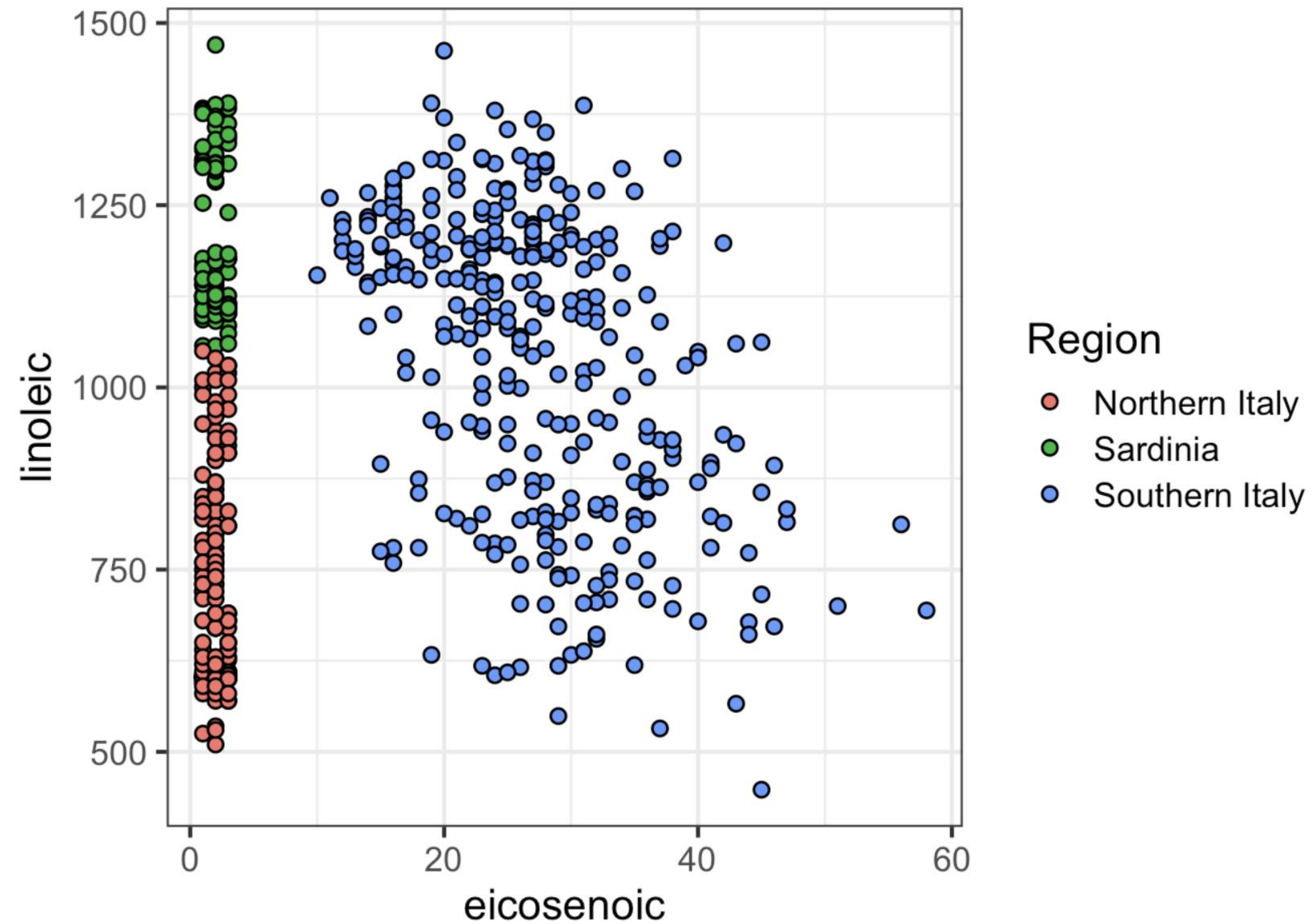
Making predictions

The way decision trees make predictions depends on the type of task.

- Classification: predicts the most common category in the final group/leaf
- Regression: predicts the **average** value in the final group/leaf

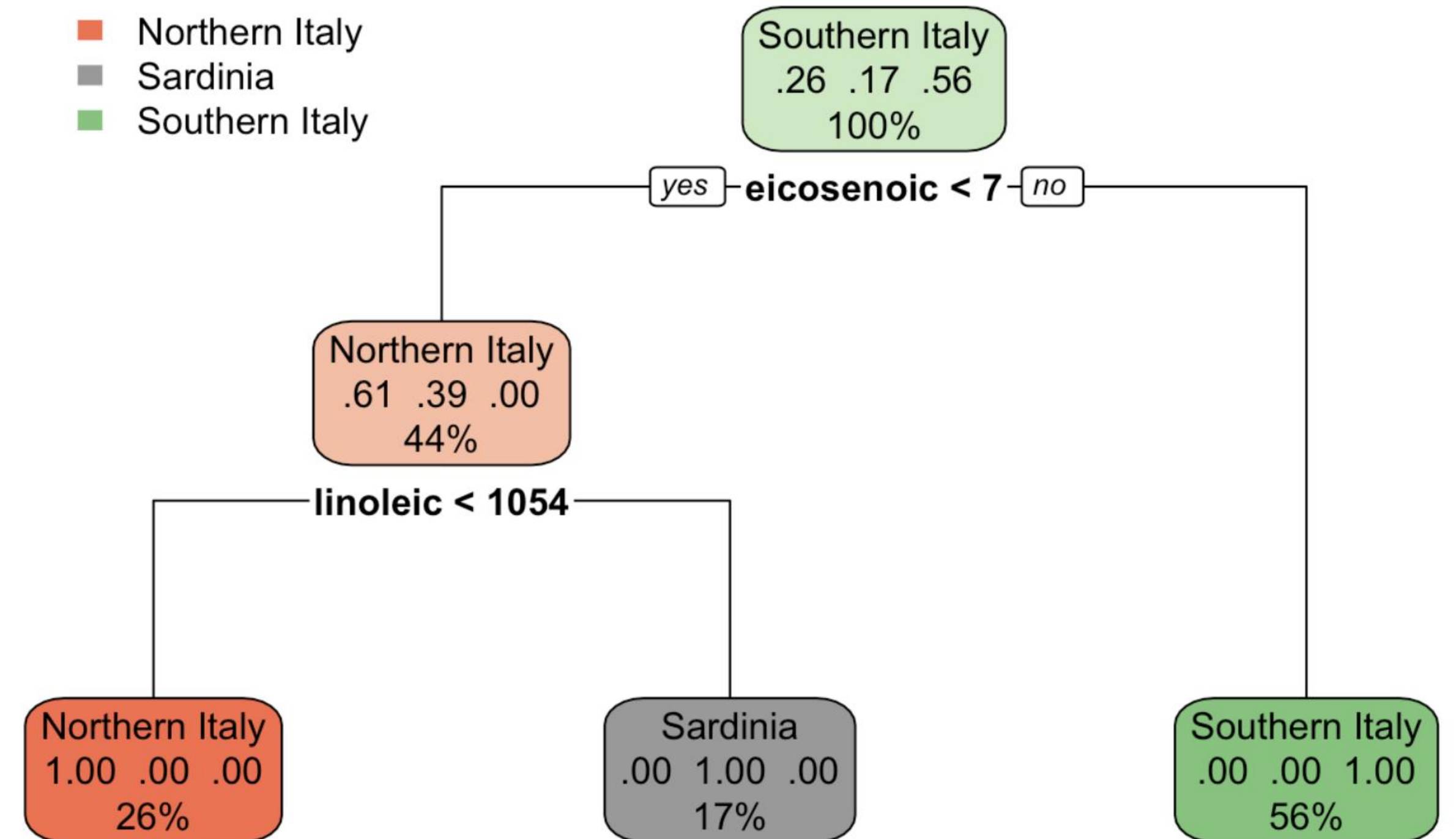
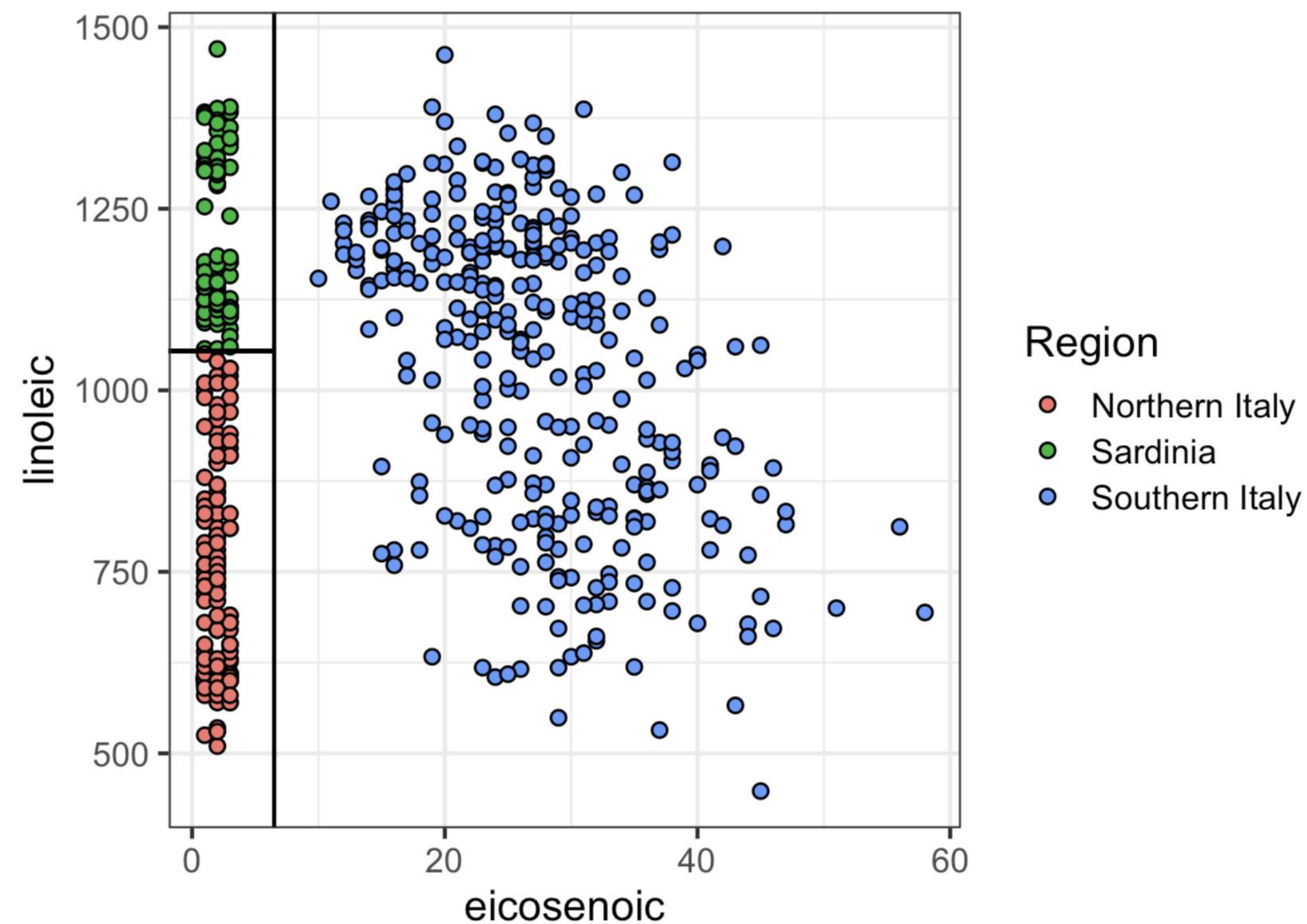


Example: olives



Example: olives

```
p <- p + geom_vline(xintercept = 6.5) +  
  geom_segment(x = -2, y = 1053.5, xend = 6.5, yend = 1053.5)  
p
```



Example: Boston Housing

The **Boston Housing** dataset contains information collected by the U.S Census Service concerning housing in the area of Boston, MA in the 1970s. There are 14 variables total. Below are details about a few of them.

- **medv**: median value of owner-occupied homes in \$1000's
- **lstat**: % of individuals with lower socioeconomic status
- **rm**: average number of rooms per dwelling
- **dis**: weighted distances to five Boston employment centres
- **nox**: nitric oxides concentration (parts per 10 million)
- **crim**: per capita crime rate by town

The **tree** function

Task: predict the median house price (**medv**)

- This is a regression task since **medv** is continuous
- Use the **tree** function from the **tree** package to train a decision tree
- The **.** means to include all other variables in the dataset as predictors

```
library(tree)
library(MASS)
library(caret)

set.seed(1)

# Randomly sample 70% of the data for training
train_index <- createDataPartition(y = Boston$medv, p = 0.7, list = FALSE)

train_set <- Boston[train_index, ]
test_set  <- Boston[-train_index, ]

# Fit a regression tree using all of the available predictors
fit = tree(medv ~ ., data = train_set)

# Print a summary of the tree
summary(fit)
```

Tree summary

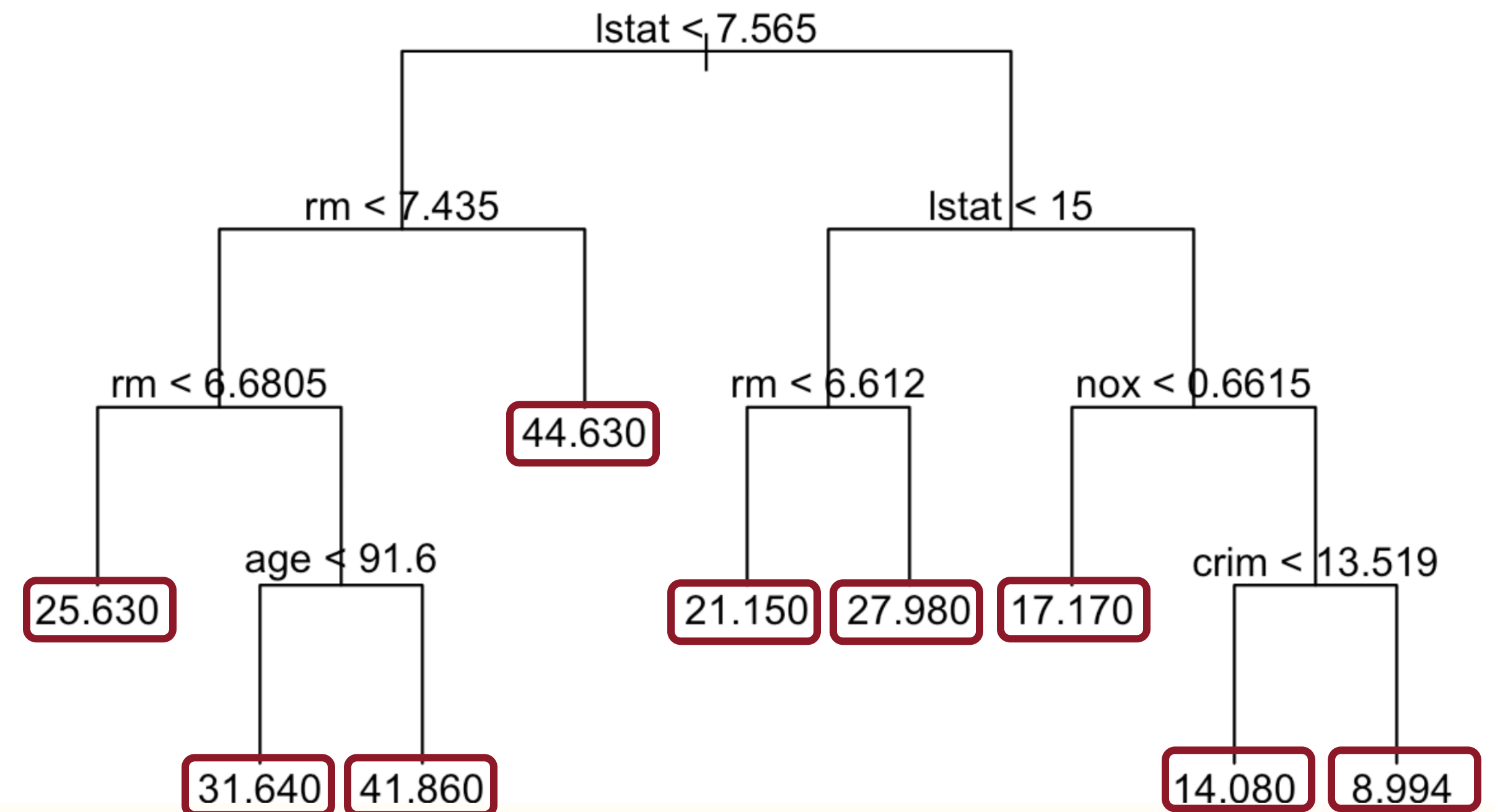
```
##  
## Regression tree:  
## tree(formula = medv ~ ., data = train_set)  
## Variables actually used in tree construction:  
## [1] "lstat" "rm"      "age"      "nox"      "crim"  
## Number of terminal nodes: 9  
## Residual mean deviance: 16.86 = 5852 / 347  
## Distribution of residuals:  
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.   
## -22.7300 -1.8500 -0.3119  0.0000  1.8500 28.8500
```


Visualizing the tree

The **plot** function can be used to display the tree.

- The **text** function adds the variable names and values to the tree
- **pretty = 0** ensures full variable names are displayed
- This tree has 9 terminal nodes (boxed values)

```
plot(fit, type = "uniform")  
text(fit, pretty = 0)
```



Making predictions

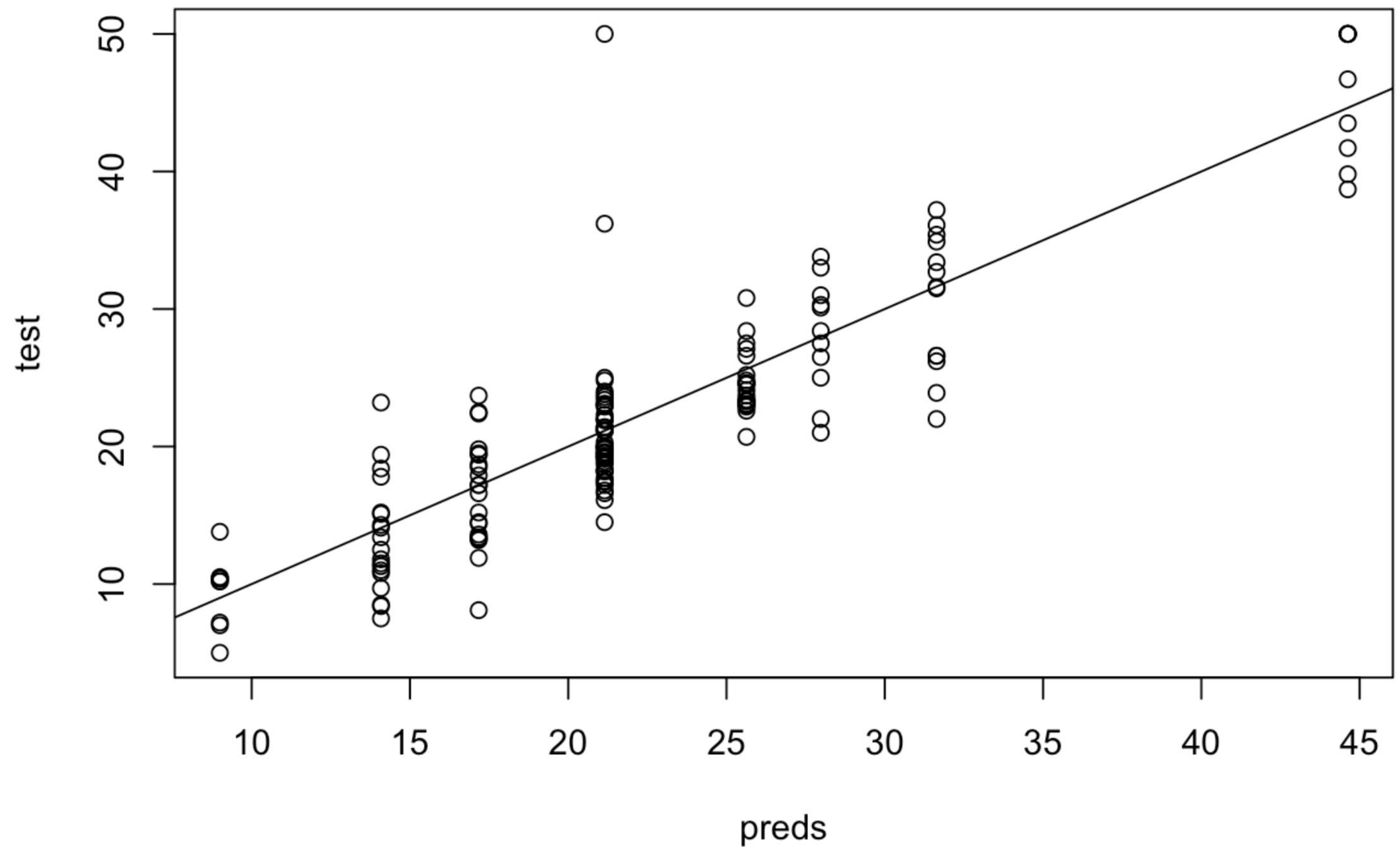
- The **predict** function is used to make predictions
- The **plot** function can be used to display the predictions and ground truth values

```
# Use tree for prediction  
preds <- predict(fit, newdata = test_set)
```

```
test <- test_set$medv  
plot(preds, test)  
abline(0,1)
```

Making predictions

- The **predict** function is used to make predictions
- The **plot** function can be used to display the predictions and ground truth values



Evaluating performance

Performance metrics are different for continuous outcomes.

- Mean squared error (MSE)
- Root mean squared error (RMSE)
- RMSE sometimes preferred for interpretability

```
mean((preds-test)^2)
```

```
## [1] 19.10161
```

```
sqrt(mean((preds-test)^2))
```

```
## [1] 4.370539
```

Interpretation: this model leads to test predictions that are within approximately \$4,370 of the true median home value.

The **rpart** function

The **rpart** function from the **rpart** package is another way to train a decision tree.

- Automatically performs 10-fold cross-validation
- Better tree visualizations

```
library(rpart)
set.seed(1)
```

```
# Fit a regression tree using all of the available predictors
fit_rpart = rpart(medv ~ ., data = train_set)
```

```
# Print a summary of the tree
summary(fit_rpart)
```

```
## Call:
```

```
## rpart(formula = medv ~ ., data = train_set)
```

```
##      n= 356
```

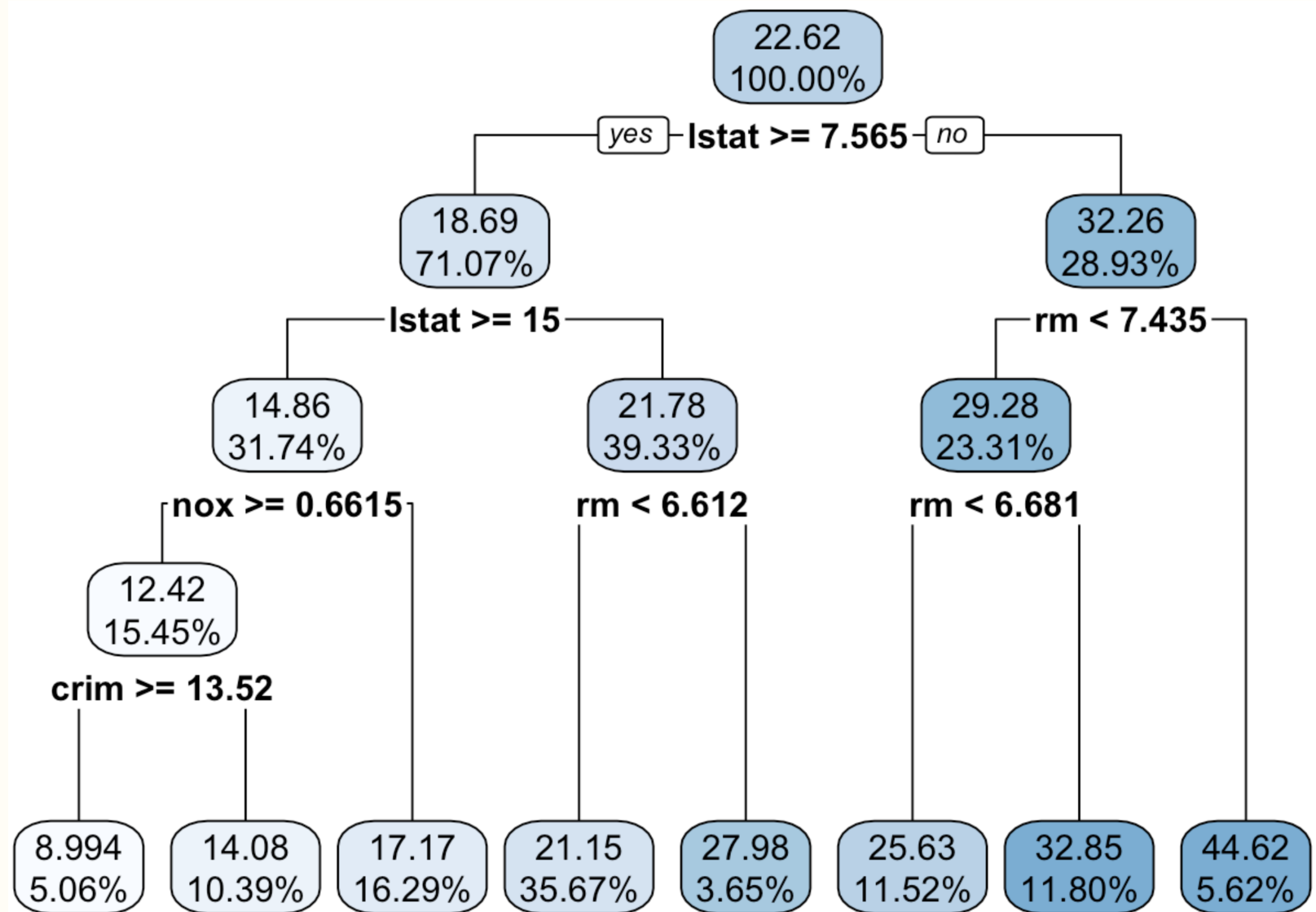
```
##
```

```
##           CP nsplit rel error    xerror    xstd
## 1 0.46224493      0 1.0000000 1.0058246 0.10004122
## 2 0.13000875      1 0.5377551 0.6214606 0.06889048
## 3 0.10281834      2 0.4077463 0.4579724 0.05832243
## 4 0.03710371      3 0.3049280 0.3665824 0.05466323
```

Visualizing the tree

```
rpart.plot(fit_rpart, digits = 4)
```

- The % indicates the percentage of training data in that leaf
- The decimal number is the mean value of the outcome for all observations in that leaf
- This tree has 8 terminal nodes



Evaluating performance

This tree has a higher MSE and RMSE but may generalize better in the future because of the use of cross-validation.

- The RMSE is only off by \$30 which isn't much comparatively

```
mean((preds-test)^2)
```

```
## [1] 19.44355
```

```
sqrt(mean((preds-test)^2))
```

```
## [1] 4.409485
```


Pruning

How does the algorithm know when to stop splitting the data?

- When adding a split does not reduce MSE
- When a leaf has too few training observations

These stopping criteria can lead to overfitting. Pruning the tree can help avoid overfitting.

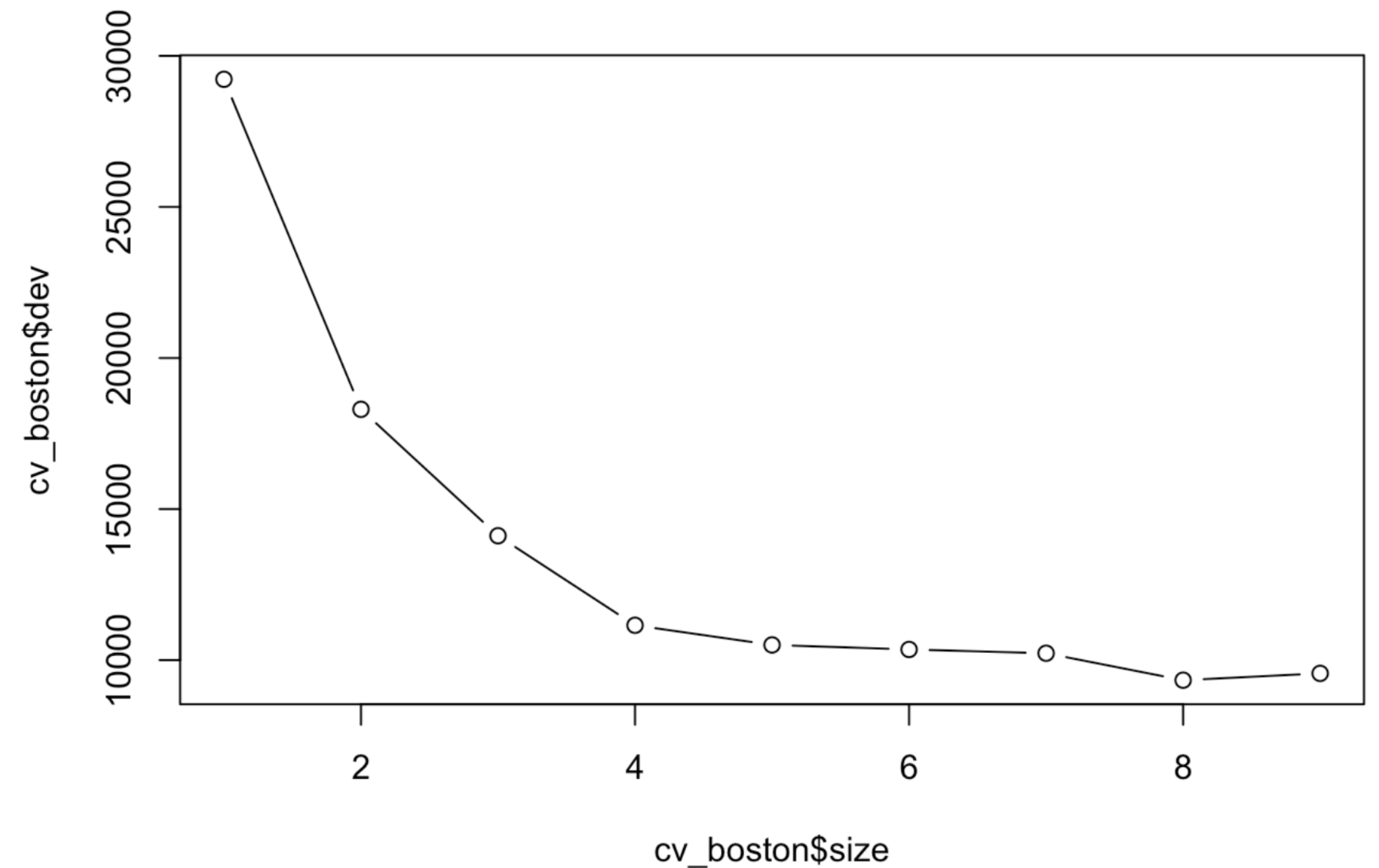


The `cv.tree` function

The `cv.tree` function performs cross-validation to determine optimal complexity of a decision tree.

- Size (how many terminal leaves) is plotted on the x-axis
- Deviance (error) is plotted on the y-axis

```
set.seed(1)
cv_boston = cv.tree(fit)
plot(cv_boston$size, cv_boston$dev, type = 'b')
```

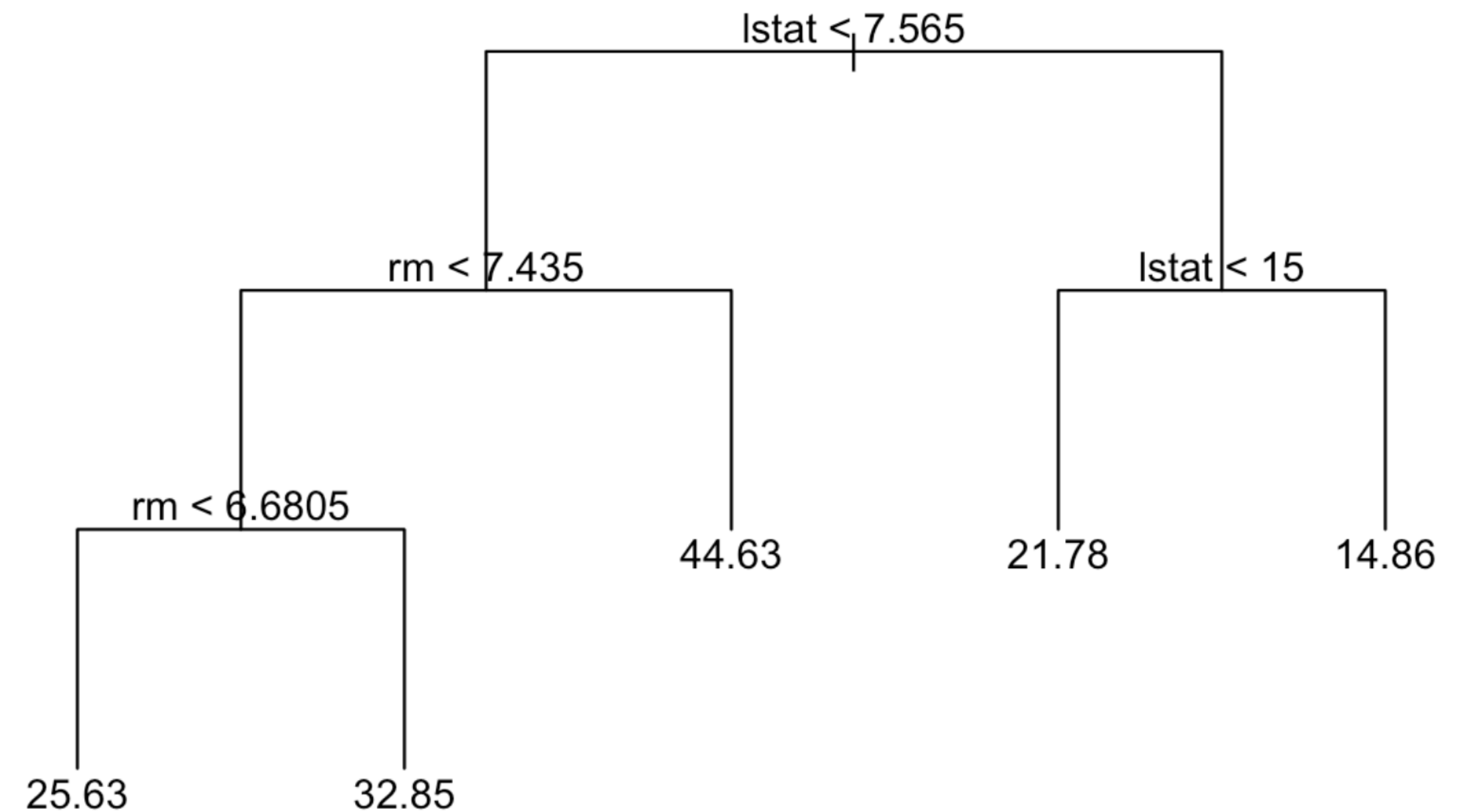


The `prune.tree` function

The `prune.tree` function can be used to trim a decision tree trained using the `tree` function.

- The `best` argument indicates how many terminal nodes to include in the tree

```
prune_boston = prune.tree(fit, best = 5)
plot(prune_boston, type = "uniform")
text(prune_boston)
```



Evaluating performance

The pruned tree does worse in terms of RMSE on the test set.

- The original tree should be used

```
preds_prune <- predict(prune_boston, newdata = test_set)
test = test_set$medv
mean((preds_prune-test)^2)
```

```
## [1] 24.85086
```

```
sqrt(mean((preds_prune-test)^2))
```

```
## [1] 4.985063
```

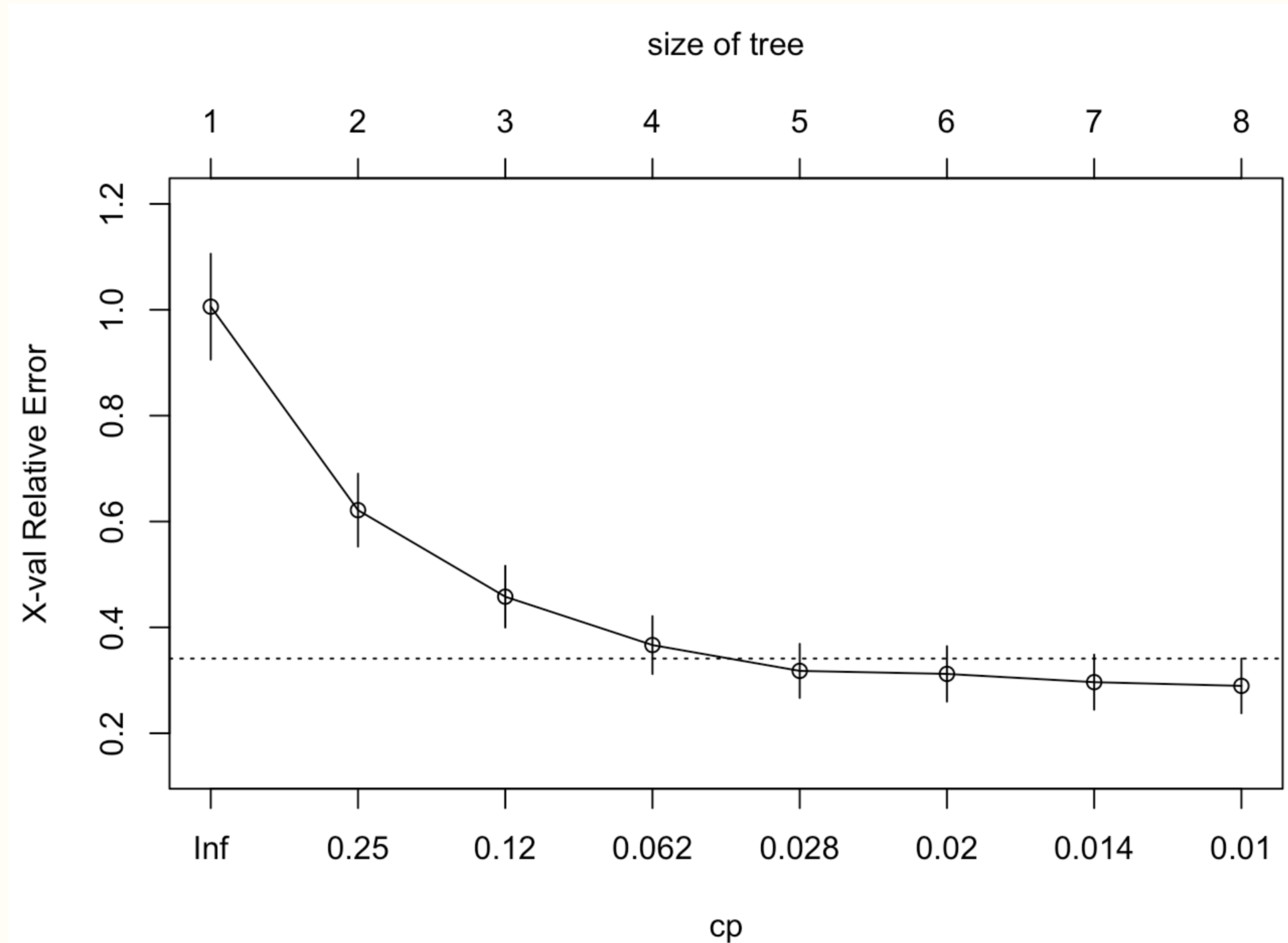

The `plotcp` function

The `plotcp` function can be used to plot the size of the tree (number of leaves) and complexity parameter (`cp`) on the x-axis and the error on the y-axis.

```
set.seed(1)
fit_rpart = rpart(medv ~ ., data = train_set)

# Use tree for prediction
preds <- predict(fit_rpart, newdata = test_set)

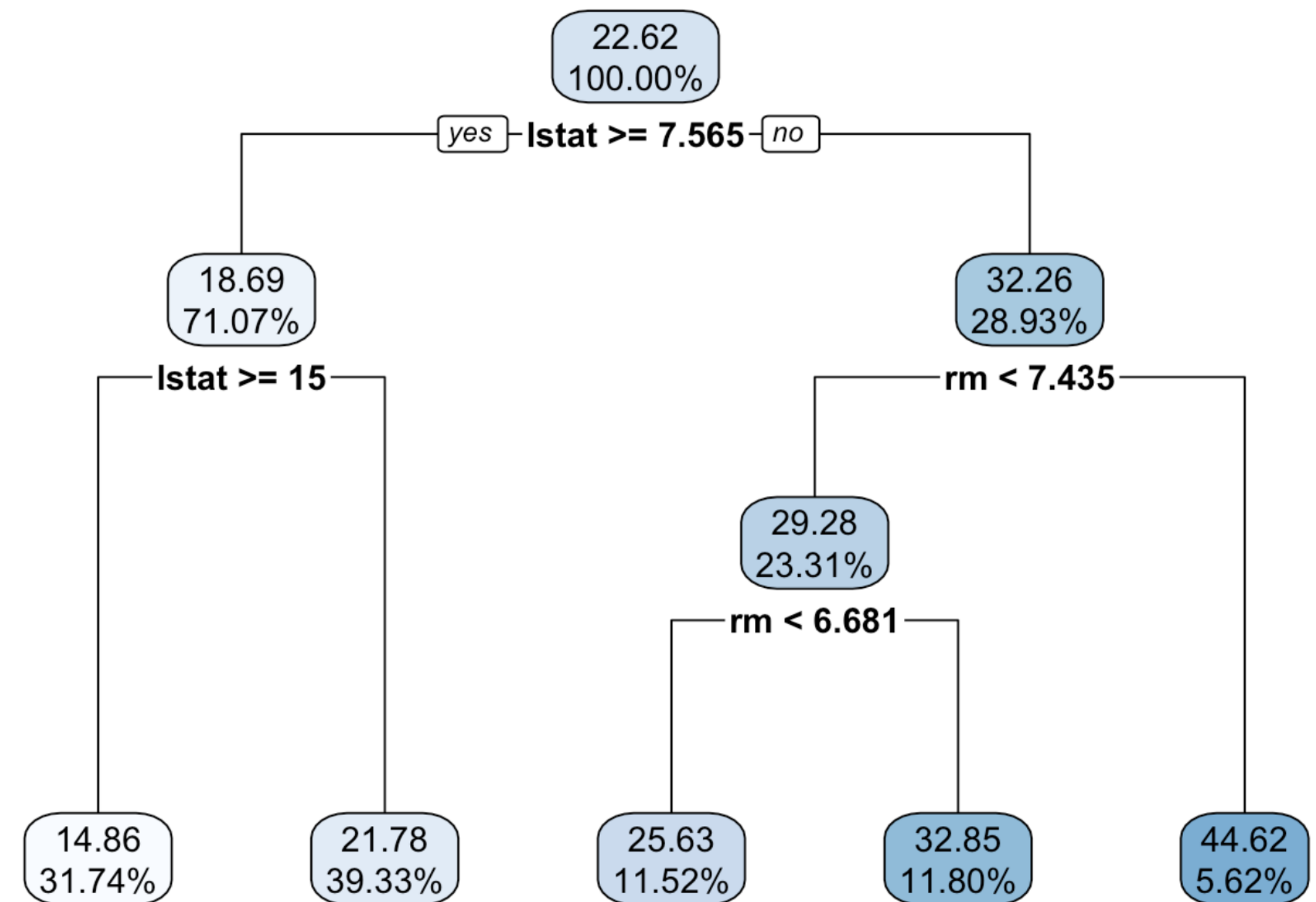
plotcp(fit_rpart)
```



The **prune** function

A pruned tree can be trained using the **prune** function and setting the complexity parameter (**cp**).

```
p <- prune(fit_rpart, cp = 0.028)
rpart.plot(p, digits = 4)
```



Evaluating performance

The pruned tree does worse than the full tree.

- The original tree should be used

```
preds <- predict(p, newdata = test_set)
```

```
mean((preds-test)^2)
```

```
## [1] 24.85086
```

```
sqrt(mean((preds-test)^2))
```

```
## [1] 4.985063
```

Model comparisons

Model	# Terminal Leaves	MSE	RMSE
Full tree with <code>tree</code> function	9	19.10	4.37
Pruned tree with <code>tree</code> function	5	24.85	4.98
Full tree with <code>rpart</code> function	8	19.44	4.41
Pruned tree with <code>rpart</code> function	5	24.85	4.98

The full tree using the `tree` function performs the best on the test set.

Decision trees summary

Pros

- Easy to explain and interpret
- Handles messy and nonlinear data well
- Requires little preprocessing

Cons

- Can be unstable – small changes in the training data may produce different trees
- Predictions can be rough or “step-like”
- Usually less accurate than more advanced models

Machine Learning

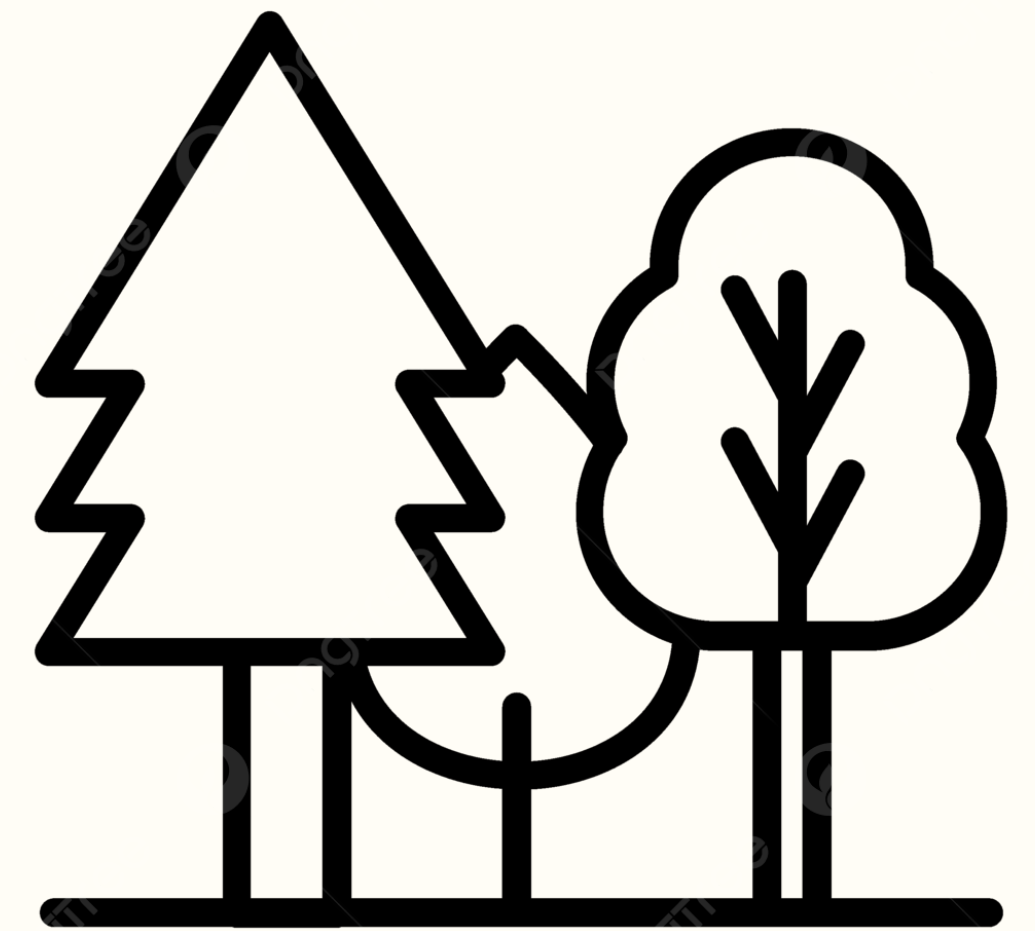
Random Forests

Heather Mattie, PhD

Random Forests

A **Random Forest (RF)** is an ensemble machine learning model that combines multiple decision trees to make more accurate and robust predictions. It is widely used for both classification and regression tasks.

- Overcomes the limitations of singular decision trees
- Ensemble method because it combines multiple decision trees to make predictions



Bagging

Bagging (bootstrap aggregation) is an ensemble technique used within Random Forests to improve predictive accuracy and reduce overfitting.

1. Build many decision trees from the training set
2. Given a new observation, calculate the prediction of the outcome for each tree
3. Aggregate the predictions; take the average for regression and the majority class for classification

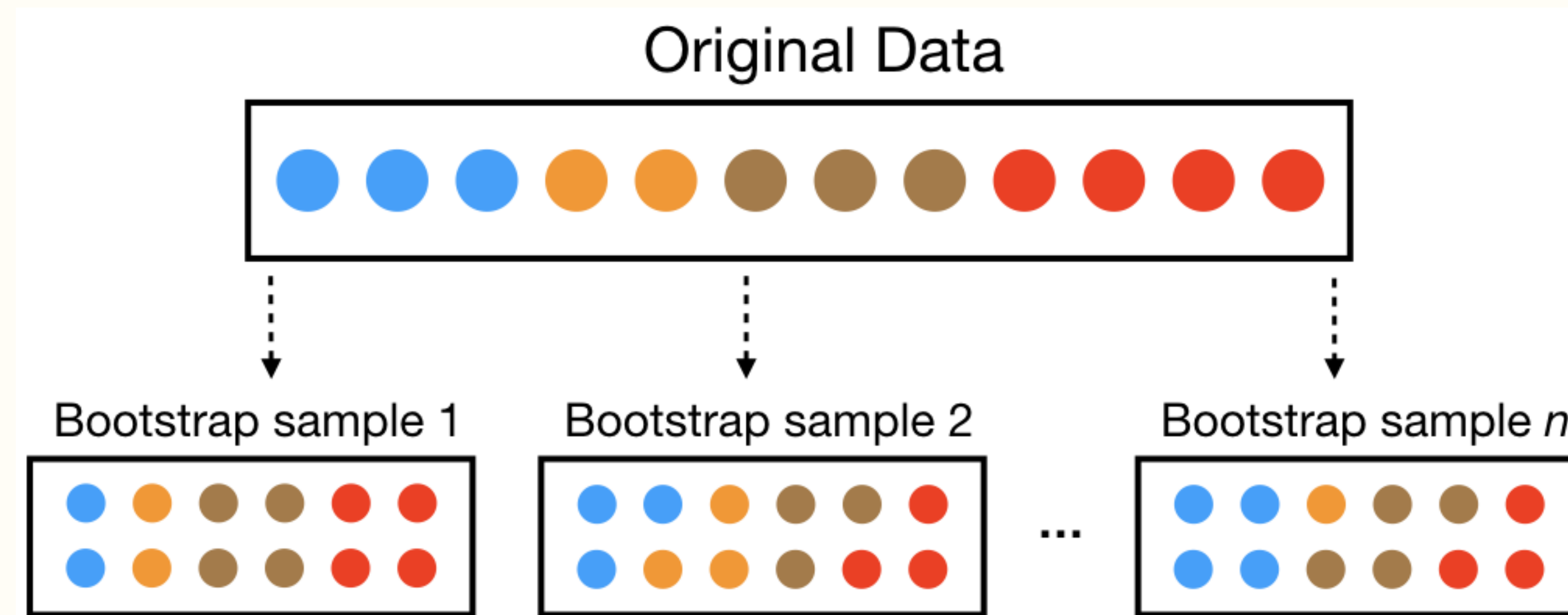
Bootstrap sampling

Bootstrap sampling creates multiple subsets of the training data by randomly sampling **with replacement** from the **original dataset**.

- Some data points may appear multiple times in a single bootstrap sample
- Some data points may be excluded entirely

Each of these distinct **bootstrap samples** is then used to train an **individual decision tree**.

Bootstrap sampling

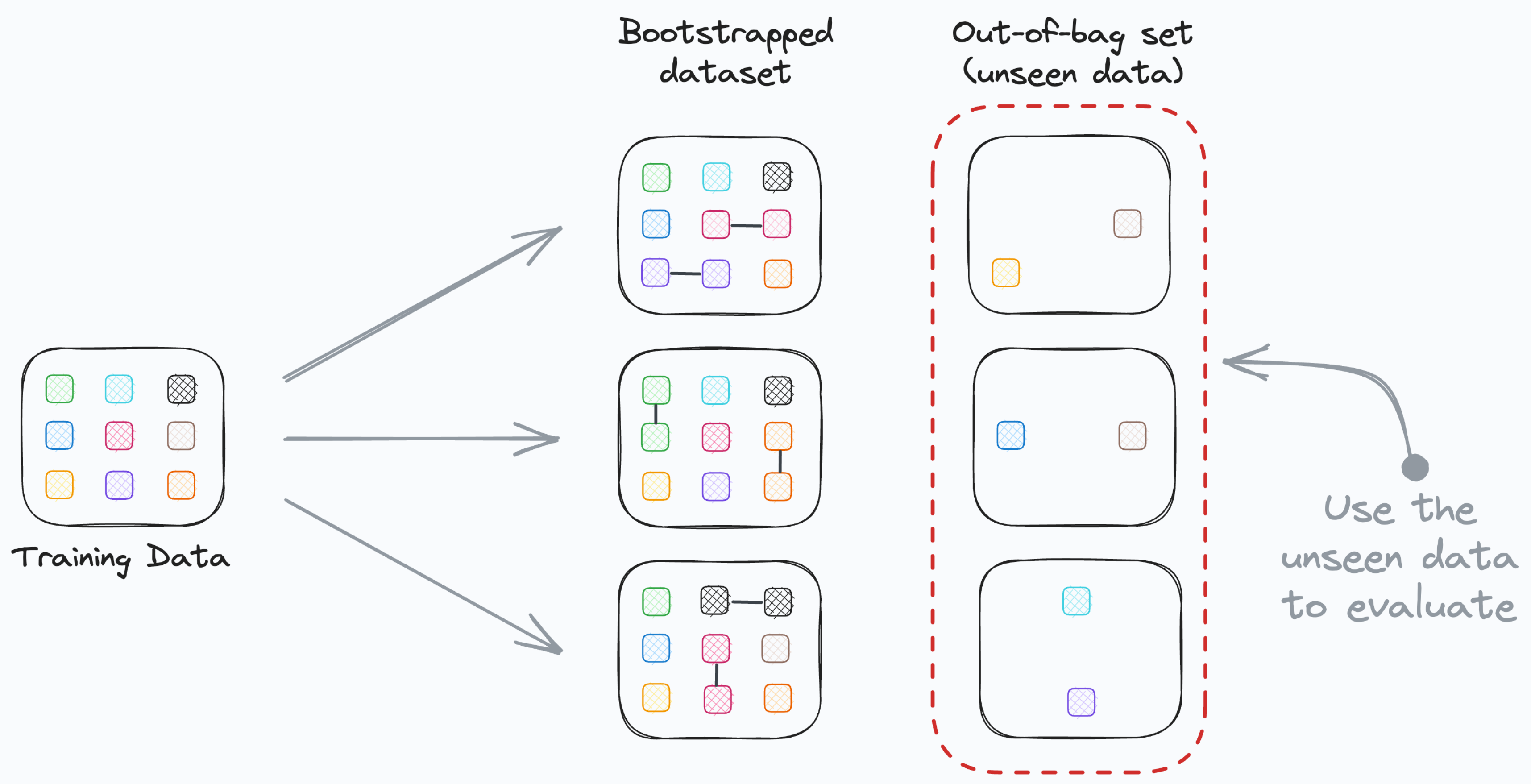


Out-of-bag (OOB)

Out-of-Bag (OOB) Samples: The instances that are *not* selected for a particular tree are called its out-of-bag (OOB) samples.

OOB Prediction: For each data point, the OOB samples are predicted by all the trees that were *not* trained on that data point. These individual OOB predictions are then aggregated (e.g., by averaging for regression or majority voting for classification) to form a final OOB prediction for that instance.

Out-of-bag (OOB)



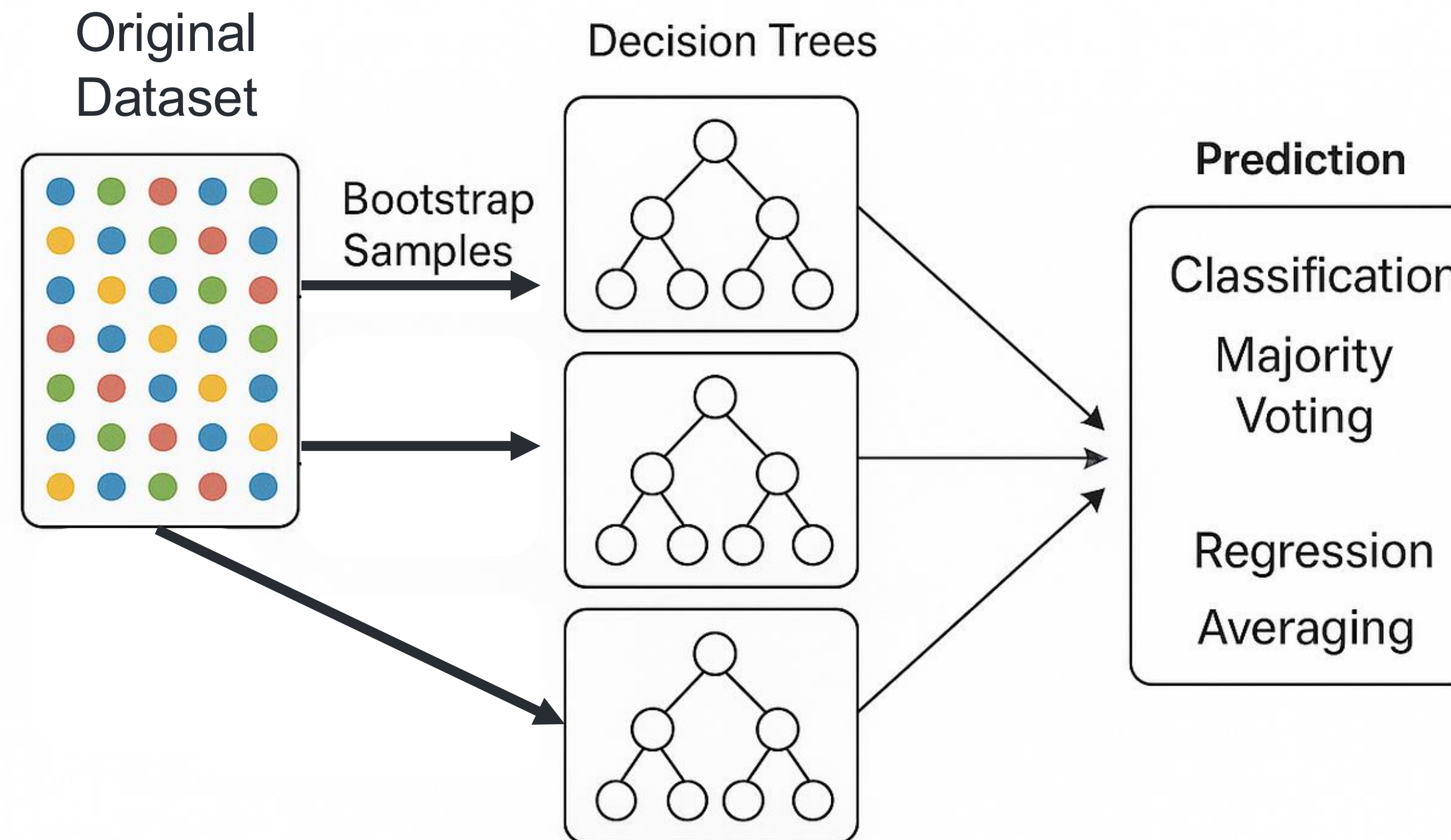
Aggregation

After each decision tree is trained on its respective bootstrap sample, the predictions from all the individual trees are combined (**aggregated**) to form a final prediction.

- For classification tasks: majority vote among the trees
- For regression tasks: the average of the predictions from all trees

Aggregation

Random Forest Regression and Classification



Feature bagging

Random Forests build upon bagging by adding an extra layer of randomness during the tree construction process. In addition to bootstrap sampling, Random Forests also randomly select a subset of features at each split point within each individual decision tree.

This “**feature bagging**” further **decorrelates** the trees, leading to even greater variance reduction and often improved predictive performance compared to traditional bagging alone.

Problems addressed

- **Variance reduction:** By training multiple trees on different subsets of the data, bagging helps to reduce the variance of the model. Each tree learns slightly different patterns due to the variations in the bootstrap samples, leading to a more robust and less sensitive overall model.
- **Overfitting mitigation:** The diversity introduced by bootstrap sampling helps to prevent individual trees from overfitting to the training data. The aggregation of predictions from diverse trees further reduces the risk of overfitting by smoothing out individual tree errors.

Example: Boston Housing

The **Boston Housing** dataset contains information collected by the U.S Census Service concerning housing in the area of Boston, MA in the 1970s. There are 14 variables total. Below are details about a few of them.

- **medv**: median value of owner-occupied homes in \$1000's
- **lstat**: % of individuals with lower socioeconomic status
- **rm**: average number of rooms per dwelling
- **dis**: weighted distances to five Boston employment centres
- **nox**: nitric oxides concentration (parts per 10 million)
- **crim**: per capita crime rate by town

The `randomForest` Function

The `randomForest` function from the `randomForest` package trains Random Forest models.

- **`mtry`**: number of predictors (p) sampled for splitting each node, default is $p/3$ for regression and $\sqrt{p}$ for classification
- **`ntree`**: number of trees grown, default is 500
- **`nodesize`**: minimum size of terminal nodes (how many observations in each terminal leaf)

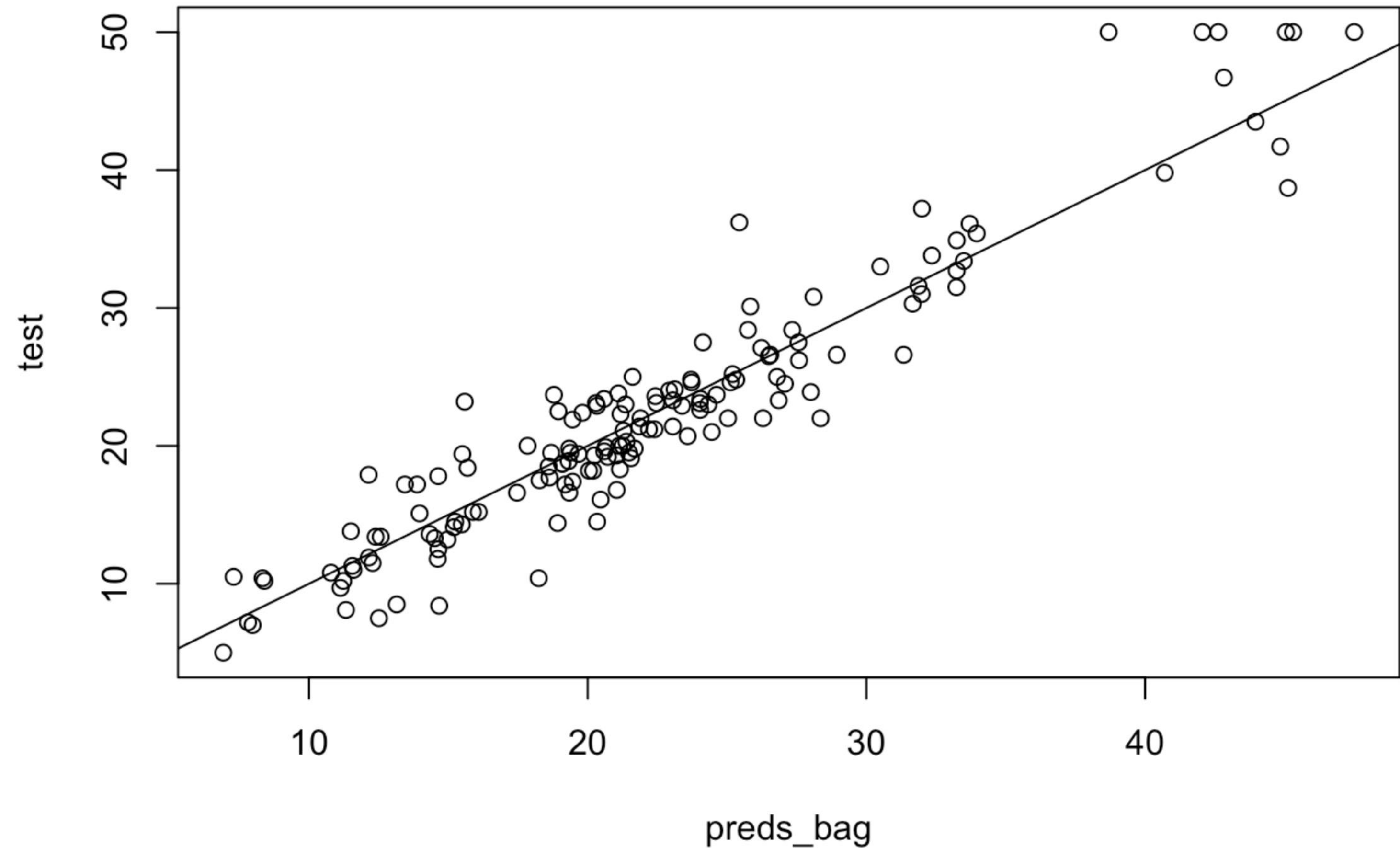
```
library(randomForest)
library(MASS)
fit_bag <- randomForest(medv ~ ., data = train_set, mtry = 13)
fit_bag
```

```
##
## Call:
## randomForest(formula = medv ~ ., data = train_set, mtry = 13)
##               Type of random forest: regression
##               Number of trees: 500
## No. of variables tried at each split: 13
##
##               Mean of squared residuals: 13.68988
##               % Var explained: 83.29
```


Making predictions

- The **predict** function is used to make predictions
- The **plot** function can be used to display the predictions and ground truth values

```
preds_bag = predict(fit_bag, newdata = test_set)
plot(preds_bag, test)
abline(0,1)
```



Evaluating performance

- The MSE/RMSE are much lower compared to the single decision trees.
- This model leads to test predictions that are within approximately \$3,021 of the true median home value.

```
mean((preds_bag - test)^2)
```

```
## [1] 9.12901
```

```
sqrt(mean((preds_bag - test)^2))
```

```
## [1] 3.021425
```

Changing `mtry`

Reducing the size of the subset of predictors chosen for each split in a tree can help prevent overfitting.

Here, it does not improve performance on the test set.

```
fit_rf <- randomForest(medv ~ ., data = train_set, mtry = 6)
preds_rf = predict(fit_rf, newdata = test_set)
mean((preds_rf - test)^2)
```

```
## [1] 10.00925
```

```
sqrt(mean((preds_rf - test)^2))
```

```
## [1] 3.16374
```

Comparing performance

Model	# Terminal Leaves	MSE	RMSE
Full tree with <code>tree</code> function	9	19.10	4.37
Pruned tree with <code>tree</code> function	5	24.85	4.98
Full tree with <code>rpart</code> function	8	19.44	4.41
Pruned tree with <code>rpart</code> function	5	24.85	4.98
Random Forest with <code>mtry</code> = 13	-	9.13	3.02
Random Forest with <code>mtry</code> = 6	-	10.01	3.16

Random Forest summary

Pros

- High accuracy
- Handles complex relationships automatically
- Resistant to noise and outliers

Cons

- Harder to interpret than one decision tree
- More computationally intensive